

SISTEMA DE GERENCIAMENTO ESPACIAL

Componentes: Eduardo Ortiz de Tunes Ramos, Felipe Andriotti Ferreira e Rafael Lopes Bermudes

1. VISÃO GERAL

O sistema é uma aplicação de terminal que consulta dados do universo Star Wars, organiza os nomes dos personagens em uma árvore Trie e permite transmitir os dados dos planetas usando codificação de Huffman.

O programa é iniciado por (sistema/main.py). Na inicialização, ele consulta os recursos da API, cria os índices necessários e apresenta um menu com as operações disponíveis.

2. FONTE DE DADOS

2.1 API ESCOLHIDA

Foi escolhida a SWAPI (<https://swapi.info/>), uma API pública que fornece dados estruturados sobre personagens, planetas e outros elementos de Star Wars.

2.2 JUSTIFICATIVA DA ESCOLHA

A API foi escolhida porque:

- Disponibiliza respostas em JSON, facilitando o processamento.
- Possui recursos separados para personagens, planetas, filmes, veículos e naves.

2.3 ENDPOINTS UTILIZADOS

A URL base utilizada é:

- <https://swapi.info/api/>

O sistema utilizou os seguintes endpoints:

- **/people/** (informações dos personagens)
- **/planets/** (informações dos planetas)
- **/films/** (informações dos filmes)
- **/vehicles/** (informações dos veículos)
- **/starships/** (informações das naves)

2.4 EXEMPLOS DE REQUISIÇÕES

A requisição em python foi feita da seguinte forma:

```
def get_all_info(category):  
    url = f"{base_url}{category}/"  
    response = requests.get(url)  
  
    if response.status_code == 200:  
        data = response.json()  
        return data  
    else:  
        print(f"Falha ao coletar os dados. Erro: {response.status_code}")  
        return None
```

2.5 ESTRUTURA DOS DADOS OBTIDOS

Cada personagem é um JSON com campos como:

```
{  
    "name": "Luke Skywalker",  
    "height": "172",  
    "mass": "77",  
    "hair_color": "blond",  
    "skin_color": "fair",  
    "eye_color": "blue",  
    "birth_year": "19BBY",  
    "gender": "male",  
    "homeworld": "https://swapi.info/api/planets/1",  
    "films": [  
        "https://swapi.info/api/films/1",  
        "https://swapi.info/api/films/2",  
        "https://swapi.info/api/films/3",  
        "https://swapi.info/api/films/6"
```

```

    ],
    "species": [],
    "vehicles": [
        "https://swapi.info/api/vehicles/14",
        "https://swapi.info/api/vehicles/30"
    ],
    "starships": [
        "https://swapi.info/api/starships/12",
        "https://swapi.info/api/starships/22"
    ],
    "created": "2014-12-09T13:50:51.644000Z",
    "edited": "2014-12-20T21:17:56.891000Z",
    "url": "https://swapi.info/api/people/1"
}

```

Planetas, filmes, veículos e naves são armazenados em dicionários cujo índice e a URL do recurso. Assim, as URLs presentes no objeto de personagem podem ser convertidas nos nomes legíveis exibidos pelo menu.

3. MODELAGEM

3.1 REPRESENTAÇÃO DOS ELEMENTOS

Elemento	Representação	Origem
Personagem	dicionário (JSON da SWAPI), armazenado na lista <code>people_info</code>	<code>/people/</code>
Planeta, filme, veículo, nave	dicionário com chave = URL e valor = nome (ou título para os filmes)	<code>/planets/</code> , <code>/films/</code> , <code>/vehicles/</code> , <code>/starships/</code>
Nome de personagem	Caminho na Trie, um nó por caractere	Campo "name"
Nó da Trie	Objeto "TrieNode"	Construído pelo sistema
Nó da árvore Huffman	Objeto "HuffmanNode"	Construído pelo sistema

3.2 ATRIBUTOS UTILIZADOS

Personagem:

- name: chave de inserção e busca na Trie.
- homeworld, films, vehicles, starships: URLs, convertidas em nomes pelos dicionários.
- height, mass, hair_color, skin_color, eye_color, birth_year, gender: usados na exibição.

TrieNode:

- children: lista de 256 posições, indexada por ord(caractere). Cada posição guarda o próximo nó ou None.
- is_end_of_word: indica que o caminho até o nó forma um nome completo.
- original_word: nome com a capitalização original, usado na exibição.

HuffmanNode:

- freq: frequência do caractere (folha) ou soma das frequências dos filhos (nó interno).
- data: nas folhas é o caractere já nos nós internos é a soma das frequências em string.
- left, right: filho esquerdo e direito.

3.3 OPERAÇÕES IMPLEMENTADAS

API

- get_all_info(category): faz o GET no endpoint e retorna a lista de recursos.

Trie

- insert(word): insere o nome em minúsculas e guarda a grafia original.
- search_word(word): verifica se o nome existe e informa os nós visitados.
- search_prefix(prefix): desce até o prefixo e coleta os nomes da subárvore.
- dfs(node, results): percurso em profundidade auxiliar, que também conta nós visitados.

Huffman

- count_freq(message): preenche freq_map.
- create_huffman_tree(): constrói a árvore com PriorityQueue.
- set_bit_code(node, bit_str): gera encode_map (0 à esquerda, 1 à direita).
- post_in_traversal(node): gera os percursos em ordem simétrica e pós-ordem.
- encode(message): executa o pipeline completo e retorna a string de bits.
- get_huffman_data(): expõe tabelas e percursos para o menu.

Menu

- Opções 1 a 5: busca por nome, busca por planeta, busca por prefixo, transmissão de informações planetárias e saída.

3.4 DECISÕES DE PROJETO

- **Organização em módulos:** (main.py) cuida da API e do menu, e as estruturas ficam em (estruturas/trie.py) e (estruturas/huffman.py).
- **Trie com vetor de 256 posições:** Cada nó da Trie possui um vetor com 256 posições para representar caracteres ASCII, evitando o uso de dicionário.
- **Dados em memória:** cada endpoint é consultado uma única vez, na inicialização. A SWAPI devolve a lista completa em uma só resposta, sem paginação.

4. ESTRUTURA DE DADOS: TRIE

4.1 JUSTIFICATIVA DA ESCOLHA

Os nomes dos personagens são sequências de caracteres e possuem muitos prefixos compartilhados. A Trie representa cada caractere em um nível da árvore, tornando mais fácil localizar um nome completo ou todos os nomes que começam com determinado prefixo.

4.2 OPERAÇÕES E COMPLEXIDADE

Considerando L o tamanho do nome ou prefixo, V o número de nós da Trie e R o número de nós visitados na subárvore do prefixo.

- `insert(word)`: $O(L)$. Percorre cada caractere, criando o nó quando ele não existe.
- `search_word(word)`: $O(L)$. Para antes se o caminho não existe.
- `search_prefix(prefix)`: $O(L + R)$. Desce até o prefixo e executa a DFS.
- `dfs(node, results)`: $O(R)$. Visita os nós alcançáveis a partir do prefixo.
- **Memória:** $O(V * 256)$. Cada nó possui 256 referências.

O acesso a um filho é $O(1)$, em troca da velocidade, o vetor usa mais memória do que uma estrutura que guarda apenas os filhos existentes (como um dicionário).

4.3 INTEGRAÇÃO COM O SISTEMA

Depois de carregar /people/, o programa executa `people_trie.insert` para cada nome. A opção 1 chama `search_word`, e a opção 3 chama `search_prefix`. Quando a busca por nome tem sucesso, o objeto completo da pessoa é localizado na lista retornada pela API, por varredura linear de custo $O(n)$, para que seus atributos e relacionamentos sejam exibidos. A opção 2 (planeta natal) não usa a Trie e é linear.

5. ANÁLISE AMORTIZADA DO CRESCIMENTO DA TRIE

5.1 PROCEDIMENTO ANALISADO

O procedimento analisado é a sequência de inserções dos nomes recebidos em /people/. Como o vetor de 256 filhos de cada nó tem tamanho fixo, o crescimento dinâmico da Trie ocorre pela criação de novos nós, quando um caractere ainda não existe no caminho, aloca-se um TrieNode com vetor de 256 posições. É essa alocação que torna uma inserção cara.

5.2 PIOR CASO

Para cada caractere processado em uma inserção, o custo é 1 se o nó filho já existe (apenas o acesso) e 257 se ele precisa ser criado (1 do acesso + 256 da alocação do vetor). O pior caso de uma inserção ocorre quando todos os caracteres criam nós, e o custo é então $257 \cdot L_i$, onde L_i é o tamanho do nome inserido.

Aplicar esse limite a cada uma das m inserções resulta em 257 vezes a soma dos tamanhos de todos os nomes. Esse limite é demasiadamente conservador, porque a alocação só ocorre para o sufixo novo de cada nome, e os prefixos já existentes são reaproveitados. Nomes como “Darth Vader” e “Darth Maul” compartilham o prefixo “darth”, e só o restante exige novos nós.

5.3 MÉTODO CONTÁBIL

Cobra-se 257 unidades por caractere processado. Uma unidade paga o acesso ao nó, e as 256 restantes ficam como crédito para pagar a alocação, caso o nó precise ser criado.

- Se o nó precisa ser criado, o crédito paga a alocação, e o saldo daquele caractere fica zerado.
- Se o nó já existe, a alocação não ocorre, e o crédito não é gasto e permanece como saldo.

Como nenhuma alocação é paga sem que o crédito correspondente tenha sido cobrado antes, o saldo nunca fica negativo. Portanto, o custo real total das m inserções nunca excede o custo amortizado total, que é 257 vezes a soma dos tamanhos dos nomes inseridos.

O custo amortizado de cada inserção é $257 \cdot L_i$, ou seja, $O(L_i)$ e depende apenas do tamanho do nome.

5.4 CONCLUSÃO

O limite tradicional $O(n)$, que sugere percorrer os registros a cada inserção, é demasiadamente conservador, a inserção percorre apenas os caracteres do nome, e a alocação de cada nó é paga uma única vez, de modo que os prefixos compartilhados não geram novo custo. Em uma sequência prolongada de inserções, o custo médio por operação permanece proporcional ao comprimento do nome, sem picos.

6. ALGORITMO GULOSO: HUFFMAN

6.1 DEFINIÇÃO DO PROBLEMA

Dada a representação textual dos planetas, o sistema precisa transformá-la em uma sequência binária menor que a representação direta de 8 bits por caractere, quando a distribuição de frequências permitir. O código deve ser prefix-free, para que nenhum código seja prefixo de outro.

6.2 ESTRATÉGIA ADOTADA

1. Contar a frequência de cada caractere.
2. Criar um nó para cada caractere e inseri-lo em uma fila de prioridade ordenada pela frequência.
3. Retirar os dois nós de menor frequência.
4. Criar um nó pai com a soma das frequências e reinseri-lo na fila.
5. Repetir até restar apenas a raiz.
6. Percorrer a árvore, aresta esquerda recebe 0 e aresta direita recebe 1. Se houver um único caractere distinto, ele recebe o código "0".
7. Substituir cada caractere da mensagem pelo seu código.

6.3 JUSTIFICATIVA DO CRITÉRIO DE ESCOLHA

Caracteres mais frequentes devem receber códigos menores, e os mais raros podem receber códigos longos, pois assim o total de bits da mensagem diminui. Por isso, a decisão local do algoritmo é sempre combinar os dois nós de menor frequência. Essa escolha é correta porque existe uma árvore ótima em que os dois símbolos menos frequentes são folhas irmãs no nível mais profundo, já que trocá-los de lugar com símbolos mais frequentes nunca aumenta o custo total. Além disso, ao fundir esses dois símbolos em um só, de frequência igual à soma das duas, obtém-se um problema menor do mesmo tipo, e uma solução ótima desse problema menor gera uma solução ótima do problema original. Por indução, a estratégia produz uma árvore de custo mínimo entre os códigos binários livres de prefixo.

6.4 ANÁLISE DA SOLUÇÃO PRODUZIDA

Seja N o tamanho da mensagem e K o número de caracteres distintos:

- contagem de frequências: $O(N)$;
- construção da árvore: $O(K \log K)$;
- geração dos códigos e percursos: $O(K)$;
- codificação da mensagem: $O(N)$.

O custo total é $O(N + K \log K)$, e a memória é $O(N + K)$, considerando a mensagem codificada, a tabela de frequências e a árvore.

Na execução com os dados da SWAPI, a mensagem tem 27.150 caracteres, com 50 símbolos distintos. A representação direta ocuparia 217.200 bits (8 bits por caractere), e a mensagem codificada ocupou cerca de 132.058 bits, o que resulta em uma taxa de compressão de 39,20%, com média de aproximadamente 4,86 bits por caractere.

O aspas simples é o caractere mais frequente (3.350 ocorrências), seguido do espaço (1.890), pois o texto repete aspas, dois-pontos, vírgulas e as chaves dos campos. Esses caracteres receberam os códigos mais curtos: 3 bits para a aspas simples ("100") e 4 bits para o espaço ("1011"). Já "x" e "q", que aparecem uma única vez cada, receberam os códigos mais longos, com 13 bits. Isso confirma o critério guloso: quanto maior a frequência, menor o código.

7. EXECUÇÃO

Ao executar o programa (sistema/main.py), um menu interativo é apresentado:

1. Buscar pessoa por nome:
 - Consulta um personagem pelo nome completo e exibe suas características.
2. Buscar pessoas por planeta natal
 - Lista os personagens associados ao planeta informado.
3. Buscar pessoas por prefixo do nome
 - Localiza nomes de personagens com determinado prefixo.
4. Transmitir informações planetárias
 - Converte as informações dos planetas em texto, aplica compressão utilizando o algoritmo de Huffman e exibe a mensagem codificada, tabela de frequências, árvores de Huffman (pós-ordem e ordem simétrica), códigos binários e taxa de compressão.
5. Sair
 - Encerra o programa.